

Logic and types in R

Lecture 02

Dr. Colin Rundel

Control Flow

Conditional Control Flow

Conditional execution of code blocks is achieved via `if` statements.

```
1 x = c(1, 3)
```

```
1 if (3 %in% x) {  
2   print("Contains 3!")  
3 }
```

```
[1] "Contains 3!"
```

```
1 if (5 %in% x) {  
2   print("Contains 5!")  
3 }
```

```
1 if (1 %in% x)  
2   print("Contains 1!")
```

```
[1] "Contains 1!"
```

```
1 if (5 %in% x) {  
2   print("Contains 5!")  
3 } else {  
4   print("Does not contain 5!")  
5 }
```

```
[1] "Does not contain 5!"
```

if is not vectorized

```
1 x = c(1, 3)
```

```
1 if (x == 1)
2   print("x is 1!")
```

Error in if (x == 1) print("x is 1!"): the condition has length :

```
1 if (x == 3)
2   print("x is 3!")
```

Error in if (x == 3) print("x is 3!"): the condition has length :

Note that the behavior seen above (thrown errors) is new in R 4.2, previous versions will only throw warnings (using only

Collapsing logical vectors

There are a couple of helper functions for collapsing a logical vector down to a single value: `any`, `all`

```
1 x = c(3,4,1)
```

```
1 x >= 2
```

```
[1] TRUE TRUE FALSE
```

```
1 any(x >= 2)
```

```
[1] TRUE
```

```
1 all(x >= 2)
```

```
[1] FALSE
```

```
1 x <= 4
```

```
[1] TRUE TRUE TRUE
```

```
1 any(x <= 4)
```

```
[1] TRUE
```

```
1 all(x <= 4)
```

```
[1] TRUE
```

```
1 if (any(x == 3))  
2   print("x contains 3!")
```

```
[1] "x contains 3!"
```

else if and else

```
1 x = 3
2
3 if (x < 0) {
4     "x is negative"
5 } else if (x > 0) {
6     "x is positive"
7 } else {
8     "x is zero"
9 }
```

[1] "x is positive"

```
1 x = 0
2
3 if (x < 0) {
4     "x is negative"
5 } else if (x > 0) {
6     "x is positive"
7 } else {
8     "x is zero"
9 }
```

[1] "x is zero"

if return values

R's `if` conditional statements return a value (invisibly), the two following implementations are equivalent.

```
1 x = 5
```

```
1 x = 5
```

```
1 s = if (x %% 2 == 0) {  
2   x / 2  
3 } else {  
4   3*x + 1  
5 }
```

```
1 if (x %% 2 == 0) {  
2   s = x / 2  
3 } else {  
4   s = 3*x + 1  
5 }
```

```
1 s
```

```
1 s
```

```
[1] 16
```

```
[1] 16
```

Exercise 2

Take a look at the following code below on the left, without running it in R what do you expect the outcome will be for each call on the right?

```
1 f = function(x) {  
2   # Check small prime  
3   if (x > 10 || x < -10) {  
4     stop("Input too big")  
5   } else if (x %in% c(2, 3,  
6     cat("Input is prime!\n")  
7   } else if (x %% 2 == 0) {  
8     cat("Input is even!\n")  
9   } else if (x %% 2 == 1) {  
10    cat("Input is odd!\n")  
11  }  
12 }
```

```
1 f(1)  
2 f(3)  
3 f(8)  
4 f(-1)  
5 f(-3)  
6 f(1:2)  
7 f("0")  
8 f("3")  
9 f("zero")
```


Conditionals and missing values

NAs can be particularly problematic for control flow,

```
1 if (2 != NA) {  
2   "Here"  
3 }
```

```
1 2 != NA
```

```
[1] NA
```

Error in if (2 != NA) {: missing value whe

```
1 if (all(c(1,2,NA,4) >= 1)) {  
2   "There"  
3 }
```

```
1 all(c(1,2,NA,4) >= 1)
```

```
[1] NA
```

Error in if (all(c(1, 2, NA, 4) >= 1)) {:

```
1 if (any(c(1,2,NA,4) >= 1)) {  
2   "There"  
3 }
```

```
1 any(c(1,2,NA,4) >= 1)
```

```
[1] TRUE
```

```
[1] "There"
```

Testing for NA

To explicitly test if a value is missing it is necessary to use `is.na` (often along with `any` or `all`).

```
1 NA == NA
```

```
[1] NA
```

```
1 is.na(NA)
```

```
[1] TRUE
```

```
1 is.na(1)
```

```
[1] FALSE
```

```
1 is.na(c(1,2,3,NA))
```

```
[1] FALSE FALSE FALSE TRUE
```

```
1 any(is.na(c(1,2,3,NA)))
```

```
[1] TRUE
```

```
1 all(is.na(c(1,2,3,NA)))
```

```
[1] FALSE
```